



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

2η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Εισαγωγή	2
Άσκηση 1	2
Α	2
Β	4
Γ	4
Άσκηση 2	6
Α	6
Β	6
Γ	11
Δ	11
Ε	12

Εισαγωγή

Σε αυτήν την άσκηση ο σκοπός μας είναι να βελτιστοποιήσουμε τις δυναμικές δομές δεδομένων δύο διαδικτυακών εφαρμογών, του Deficit Round Robin (DRR) και του αλγορίθμου Dijkstra με χρήση της μεθοδολογίας Dynamic Data Type Refinement. Θα βελτιστοποιήσουμε τις δυναμικές δομές δεδομένων ως προς:

- Τις προσβάσεις στην μνήμη (memory accesses) που απαιτούνται για να προσπελαστούν τα δεδομένα τους.
- ως προς την μέγιστη ποσότητα μνήμης που καταλαμβάνουν (memory footprint)

Για την άσκηση θα χρησιμοποιήσουμε την βιβλιοθήκη DDTR και τα εργαλεία Massif και Lackey της Valgrind suite.

Άσκηση 1

Στην 1η άσκηση θα βελτιστοποιήσουμε τις δυναμικές δομές δεδομένων του αλγορίθμου DRR. Η υλοποίηση του αλγορίθμου περιλαμβάνει τις δομές δεδομένων **clientList** και **pList**, όπου η 1η είναι μία δομή δεδομένων στην οποία αποθηκεύονται οι nodes και η 2η μια δομή δεδομένων που διατηρεί ο κάθε node και περιλαμβάνει τα πακέτα που θα προωθήσει. Για κάθε μία από αυτές τις δομές δεδομένων θα δοκιμάσουμε διαφορετικές υλοποιήσεις από τις παρακάτω:

- Απλά συνδεδεμένη λίστα -Single Linked List (SLL)
- Διπλά Συνδεδεμένη Λίστα - Double Linked List (DLL)
- Δυναμικό Πίνακα -Dynamic Array (ή vector) (DYN_ARR)

A)

Για να επιλέξουμε κάθε φορά την υλοποίηση για κάθε δομή δεδομένων αυτό που χρειάζεται να κάνουμε είναι να αλλάξουμε το παρακάτω κομμάτι στον κώδικα του drr.h ώστε να τρέξουμε με τις υλοποιήσεις που θέλουμε κάθε φορά:

```
C drr.h > DLL_PK
1 // #define SLL_CL
2 // #define DLL_CL
3 #define DYN_ARR_CL
4
5 // #define SLL_PK
6 #define DLL_PK
7 // #define DYN_ARR_PK
8
```

Η διαδικασία που ακολουθούμε στην συνέχεια κάθε φορά είναι η παρακάτω:

1. Compile του κώδικα:

```
gcc drr.c -o drr -pthread -lcdsl -no-pie -L../synch_implementations -I../synch_implementations
```

2. Δημιουργία trace file των προσβάσεων στην μνήμη με της χρήση του εργαλείου lackey:

```
valgrind --log-file='mem_accesses_log.txt' --tool=lackey --trace-mem=yes ./drr
```

3. Καταμέτρηση των προσβάσεων στην μνήμη από το trace file:

```
grep -c 'I\| L' mem_accesses_log.txt
```

4. Χρήση του εργαλείου massif για δημιουργία και επεξεργασία ενός log αρχείου:

```
valgrind --tool=massif ./drr
```

5. Εξαγωγή memory footprint:

```
ms_print massif.out.XXXX > mem_footprint_log.txt
```

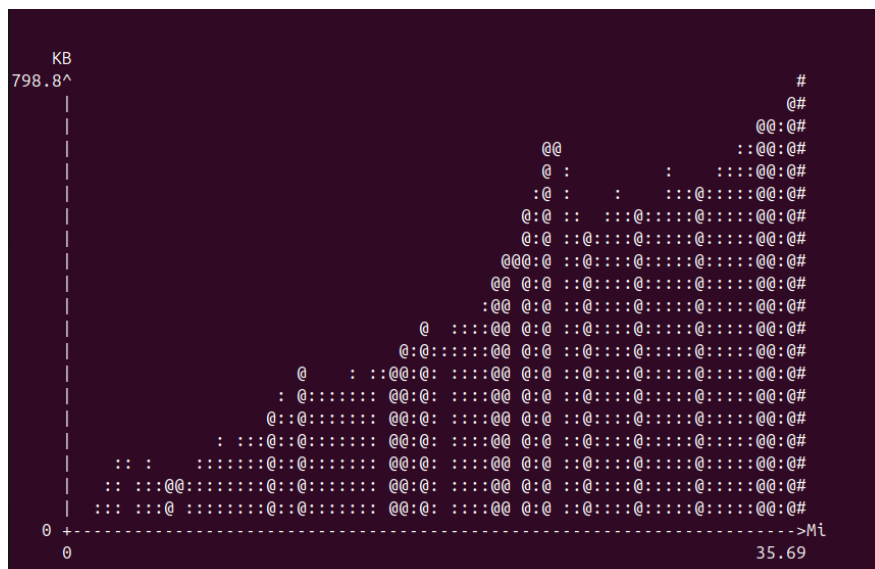
και

```
cat mem_footprint_log.txt
```

Η παραπάνω διαδικασία για την περίπτωση που οι δύο δομές υλοποιούνται με απλή συνδεδεμένη λίστα φαίνεται παρακάτω:

```
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-mem=yes ./drr
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ ls
drr drr.c drr.h mem_accesses_log.txt trace.h
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ grep -c 'I\| L' mem_accesses_log.txt
71841614
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ valgrind --tool=massif ./drr
==6660== Massif, a heap profiler
==6660== Copyright (C) 2003-2017, and GNU GPL'd, by Nicholas Nethercote
==6660== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==6660== Command: ./drr
==6660==
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ ls
drr drr.c drr.h massif.out.6660 mem_accesses_log.txt trace.h
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ ms_print massif.out.6660 > mem_footprint_log.txt
jimkam@jimkam-laptop:~/Desktop/embsys_lab2/src/DRR$ cat mem_footprint_log.txt
```

και ακολουθεί για αυτήν την περίπτωση το memory footprint:



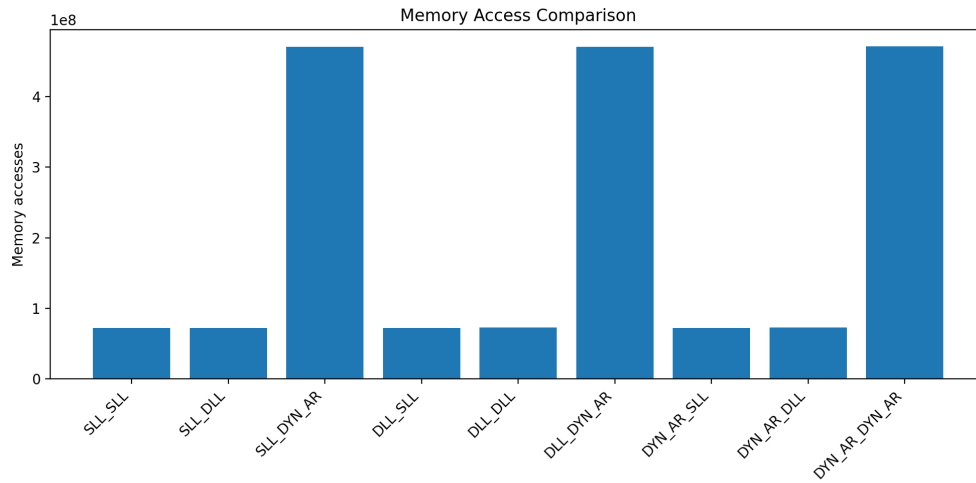
Στον παρακάτω πίνακα παρουσιάζουμε συγκεντρωτικά τα αποτελέσματα:

clientList	pList	memory accesses	memory footprint
SLL	SLL	71,841,614	798.8 KB
SLL	DLL	72,512,734	980.3 KB
SLL	DYN_AR	470,572,566	1.111 MB
DLL	SLL	71,854,278	823.0 KB
DLL	DLL	72,524,570	983.3 KB
DLL	DYN_AR	470,586,954	1.128 MB
DYN_AR	SLL	72,320,979	760.2 KB
DYN_AR	DLL	73,010,341	928.5 KB
DYN_AR	DYN_AR	471,308,480	1.075 MB

Table 1: Memory accesses και memory footprint για συνδυασμούς clientList και pList.

B)

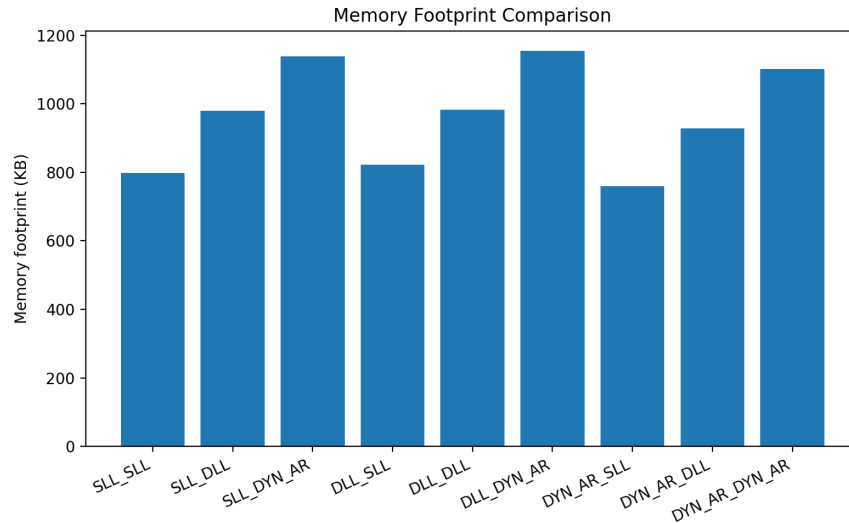
Παρακάτω παρουσιάζουμε ένα διάγραμμα για τον αριθμό των memory accesses για όλες τις πιθανές υλοποιήσεις:



Παρατηρούμε ότι από τη σκοπία των προσβάσεων στην μνήμη η χειρότερες επιλογές περιλαμβάνουν υλοποίηση της pList με δυναμικό πίνακα (λόγω resizing όταν ο πίνακας γεμίζει). Οι υπόλοιπες υλοποιήσεις δεν έχουν πολύ μεγάλες διαφορές μεταξύ τους, με την βέλτιστη να είναι αυτή στην οποία και οι δύο δομές δεδομένων υλοποιούνται με απλά συνδεδεμένη λίστα.

Γ)

Παρακάτω παρουσιάζουμε ένα διάγραμμα για το memory footprint για όλες τις πιθανές υλοποιήσεις:



Για το memory footprint παρατηρούμε ότι και εδώ οι περιπτώσεις υλοποίησης της pList με δυναμικό πίνακα έχει το χειρότερο αποτέλεσμα και μάλιστα αυτές είναι οι μόνες περιπτώσεις που το memory footprint ξεπερνάει το 1 MB. Για την ίδια υλοποίηση της clientList, η υλοποίηση της pList με απλά συνδεδεμένη λίστα έχει καλύτερο memory footprint από ότι με διπλά συνδεδεμένη λίστα. Για τις περιπτώσεις της clientList μπορούμε να παρατηρήσουμε ότι για δυναμικό πίνακα έχουμε το καλύτερο αποτέλεσμα και ακολουθεί η απλά συνδεδεμένη λίστα και η διπλά συνδεδεμένη λίστα.

Τα αποτελέσματα ανάμεσα στην SLL και την DLL είναι λογικά, αφού η 1η έχει 1 pointer για κάθε στοιχείο και η 2η 2. Στην περίπτωση της clientList παρατηρούμε διαφορετική συμπεριφορά για την περίπτωση του δυναμικού πίνακα, κάτι το οποίο προκύπτει επειδή ο δυναμικός πίνακας δεν έχει pointers και καταλαμβάνει λιγότερο χώρο και επειδή οι nodes είναι λιγότερο "δυναμικοί" (με την έννοια ότι δεν προστίθεται ή αφαιρούνται συνεχώς) από τα πακέτα παρατηρούμε το χαμηλό memory footprint.

Ο συνδυασμός που δίνει το καλύτερο memory footprint είναι η clientList με δυναμικό πίνακα και η pList με απλά συνδεδεμένη λίστα και παρουσιάζουμε και το αντίστοιχο memory footprint:



Άσκηση 2

A)

Η εκτέλεση της εφαρμογής έχει το παρακάτω αποτέλεσμα:

```
jlnkam@jlnkam-laptop:~/Desktop/embsys_lab2/src/dijkstra$ ./dijkstra input.dat
Shortest path is 1 in cost. Path is: 0 41 45 51 50
Shortest path is 0 in cost. Path is: 1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is: 2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is: 3 53
Shortest path is 1 in cost. Path is: 4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is: 5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is: 6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is: 7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is: 8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is: 9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is: 10 60
Shortest path is 5 in cost. Path is: 11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is: 12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is: 13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is: 14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is: 15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is: 16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is: 17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is: 18 15 41 45 51 68
Shortest path is 2 in cost. Path is: 19 69
```

B)

Για να εισάγουμε την βιβλιοθήκη στην εφαρμογή και να αντικαταστήσουμε στην δομή δεδομένων της τις δομές δεδομένων της βιβλιοθήκης ακολουθούμε τα παρακάτω βήματα:

1. Εισαγωγή της βιβλιοθήκης DDTR στην εφαρμογή.
2. Αντικατάσταση των δηλώσεων των δομών δεδομένων με τις δομές δεδομένων της βιβλιοθήκης.
3. Δημιουργούμε και αρχικοποιούμε τις δομές δεδομένων.
4. Αντικαθιστούμε τις συναρτήσεις enqueue και dequeue της εφαρμογής ώστε να χρησιμοποιούν τις δυναμικές δομές δεδομένων της βιβλιοθήκης.

Σημειώνουμε για αυτό το τελευταίο βήμα ότι ακολουθούμε τις οδηγίες που υπάρχουν για την προηγούμενη άσκηση και έτσι, για την συνάρτηση enqueue χρησιμοποιούμε την `qHead->enqueue(0, qHead, (void*)qNew)`; για να γίνει αυτόματη προσθήκη στην λίστα.

Για την συνάρτηση dequeue χρειαζόμαστε έναν iterator για κάθε υλοποίηση (όπως και στο `dllr`), ο οποίος θα δείχνει στην αρχή της ουράς (`iter_begin`). Στην συνέχεια, λαμβάνουμε τα δεδομένα στα οποία δείχνει ο iterator (`iter_deref`) και χρησιμοποιούμε την συνάρτηση `qHead->dequeue(0, qHead)` για να κάνουμε dequeue (αντίστοιχα πάλι με την προηγούμενη άσκηση και τις οδηγίες).

Για όλα τα παραπάνω βήματα ο τελικός κώδικας έχει την μορφή που φαίνεται παρακάτω, στον οποίο υπάρχουν σχόλια που φανερώνουν που έχουν γίνει οι αλλαγές του κάθε βήματος:

```
#include <stdio.h>
#include <stdlib.h>

#define NUM_NODES 100
#define NONE 9999

/*===== Step 1 =====*/
```

```

#define SLL
// #define DLL
// #define DYN_ARR

#if defined SLL
#include ../synch_implementations/cdsl_queue.h
#endif
#if defined DLL
#include ../synch_implementations/cdsl_deque.h
#endif
#if defined DYN_ARR
#include ../synch_implementations/cdsl_dyn_array.h
#endif
/*===== Step 1 =====*/

struct _NODE
{
    int iDist;
    int iPrev;
};
typedef struct _NODE NODE;

struct _QITEM
{
    int iNode;
    int iDist;
    int iPrev;
    struct _QITEM *qNext;
};
typedef struct _QITEM QITEM;

/*===== Step 2 =====*/
#if defined SLL
cdsl_sll *qHead;
#endif
#if defined DLL
cdsl_dll *qHead;
#endif
#if defined DYN_ARR
cdsl_dyn_array *qHead;
#endif
/*===== Step 2 =====*/

int AdjMatrix[NUM_NODES][NUM_NODES];

int g_qCount = 0;
NODE rgnNodes[NUM_NODES];
int ch;
int iPrev, iNode;
int i, iCost, iDist;

void print_path (NODE *rgnNodes, int chNode)
{
    if (rgnNodes[chNode].iPrev != NONE)
    {
        print_path(rgnNodes, rgnNodes[chNode].iPrev);
    }
    printf ( "%d ", chNode);
}

```

```

    fflush(stdout);
}

void enqueue (int iNode, int iDist, int iPrev)
{
    QITEM *qNew = (QITEM *) malloc(sizeof(QITEM));

    if (!qNew)
    {
        fprintf(stderr, "Out of memory.\n ");
        exit(1);
    }
    qNew->iNode = iNode;
    qNew->iDist = iDist;
    qNew->iPrev = iPrev;

    /*===== Step 4a =====*/
    qHead->enqueue(0, qHead, (void*)qNew);
    /*===== Step 4a =====*/

    g_qCount++;
}

void dequeue (int *piNode, int *piDist, int *piPrev)
{
    /*===== Step 4b =====*/
    #if defined SLL
    iterator_cdsl_sll it;
    #endif
    #if defined DLL
    iterator_cdsl_dll it;
    #endif
    #if defined DYN_ARR
    iterator_cdsl_dyn_array it;
    #endif

    it = qHead->iter_begin(qHead);

    QITEM *item = (QITEM*)(qHead->iter_deref(qHead, it));

    *piNode = item->iNode;
    *piDist = item->iDist;
    *piPrev = item->iPrev;

    qHead->dequeue(0, qHead);

    free(item);
    g_qCount--;
    /*===== Step 4b =====*/
}

int qcount (void)

```

```

{
    return(g_qCount);
}

int dijkstra(int chStart, int chEnd)
{

    for (ch = 0; ch < NUM_NODES; ch++)
    {
        rgnNodes[ch].iDist = NONE;
        rgnNodes[ch].iPrev = NONE;
    }

    if (chStart == chEnd)
    {
        printf( "Shortest path is 0 in cost. Just stay where you are.\n );
    }
    else
    {
        rgnNodes[chStart].iDist = 0;
        rgnNodes[chStart].iPrev = NONE;

        enqueue (chStart, 0, NONE);

        while (qcount() > 0)
        {
            dequeue (&iNode, &iDist, &iPrev);
            for (i = 0; i < NUM_NODES; i++)
            {
                if ((iCost = AdjMatrix[iNode][i]) != NONE)
                {
                    if ((NONE == rgnNodes[i].iDist) ||
                        (rgnNodes[i].iDist > (iCost + iDist)))
                    {
                        rgnNodes[i].iDist = iDist + iCost;
                        rgnNodes[i].iPrev = iNode;
                        enqueue (i, iDist + iCost, iNode);
                    }
                }
            }
        }

        printf( "Shortest path is %d in cost.  ", rgnNodes[chEnd].iDist);
        printf( "Path is:  ");
        print_path(rgnNodes, chEnd);
        printf( "\n );
    }
}

int main(int argc, char *argv[]) {

    /*===== Step 3 =====*/
    #if defined SLL
    qHead = cds1_sll_init();
    #endif
    #if defined DLL
    qHead = cds1_dll_init();

```

```

#endif
#ifdef DYN_ARR
qHead = cds1_dyn_array_init();
#endif
/*===== Step 3 =====*/

int i,j,k;
FILE *fp;

if (argc<2) {
    fprintf(stderr, Usage: dijkstra <filename>\n );
    fprintf(stderr, Only supports matrix size is #define'd.\n );

    exit(1);
}

/* open the adjacency matrix file */
fp = fopen (argv[1], r );

/* make a fully connected matrix */
for (i=0;i<NUM_NODES;i++) {
    for (j=0;j<NUM_NODES;j++) {
        /* make it more sparce */
        fscanf(fp, %d ,&k);
        AdjMatrix[i][j]= k;
    }
}

/* finds 10 shortest paths between nodes */
for (i=0,j=NUM_NODES/2;i<20;i++,j++) {
    j=j%NUM_NODES;

    dijkstra(i,j);
}
exit(0);
}

```

Κάνουμε compile την εφαρμογή όπως και στην 1η άσκηση και παρατηρούμε ότι τα αποτελέσματα μετά τις αλλαγές μας είναι σωστά:

```

jinkan@jinkan-laptop:~/Desktop/embsys_lab2/src/dijkstra$ gcc dijkstra.c -o dijk -pthread -lcdsl -no-pie -L../synchron_implementation -I../synchron_implementation
jinkan@jinkan-laptop:~/Desktop/embsys_lab2/src/dijkstra$ ./dij input.dat
Shortest path is 1 in cost. Path is: 0 41 45 51 50
Shortest path is 0 in cost. Path is: 1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is: 2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is: 3 53
Shortest path is 1 in cost. Path is: 4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is: 5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is: 6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is: 7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is: 8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is: 9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is: 10 60
Shortest path is 5 in cost. Path is: 11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is: 12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is: 13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is: 14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is: 15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is: 16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is: 17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is: 18 15 41 45 51 68
Shortest path is 2 in cost. Path is: 19 69

```

Γ)

Ακολουθώντας τον τρόπο εκτέλεσης της 1ης άσκησης και για αυτήν έχουμε την παρακάτω ροή εκτέλεσης:

```
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ gcc dijkstra.c -o dij -pthread -ldsl -no-pie -L../synch_implementations -I../synch_implementations
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-mem=yes ./dij input.dat
Shortest path is 1 in cost. Path is: 0 41 45 51 50
Shortest path is 0 in cost. Path is: 1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is: 2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is: 3 53
Shortest path is 1 in cost. Path is: 4 05 03 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is: 5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is: 6 42 00 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is: 7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is: 8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is: 9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is: 10 60
Shortest path is 5 in cost. Path is: 11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is: 12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is: 13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is: 14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is: 15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is: 16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is: 17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is: 18 15 41 45 51 68
Shortest path is 2 in cost. Path is: 19 69
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ grep -c 'I| L' mem_accesses_log.txt
149452450
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ valgrind --tool=massif ./dij input.dat
==14444== Massif, a heap profiler
==14444== Copyright (c) 2003-2017, and GNU GPL'd, by Nicholas Nethercote
==14444== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==14444== Command: ./dij input.dat
==14444==
Shortest path is 1 in cost. Path is: 0 41 45 51 50
Shortest path is 0 in cost. Path is: 1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is: 2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is: 3 53
Shortest path is 1 in cost. Path is: 4 05 03 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is: 5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is: 6 42 00 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is: 7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is: 8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is: 9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is: 10 60
Shortest path is 5 in cost. Path is: 11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is: 12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is: 13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is: 14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is: 15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is: 16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is: 17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is: 18 15 41 45 51 68
Shortest path is 2 in cost. Path is: 19 69
==14444==
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ ms_print massif.out.14444 > mem_footprint_log.txt
jinkan@jinkan-laptop:~/desktop/engsys_lab2/src/dijkstra$ cat mem_footprint_log.txt
```

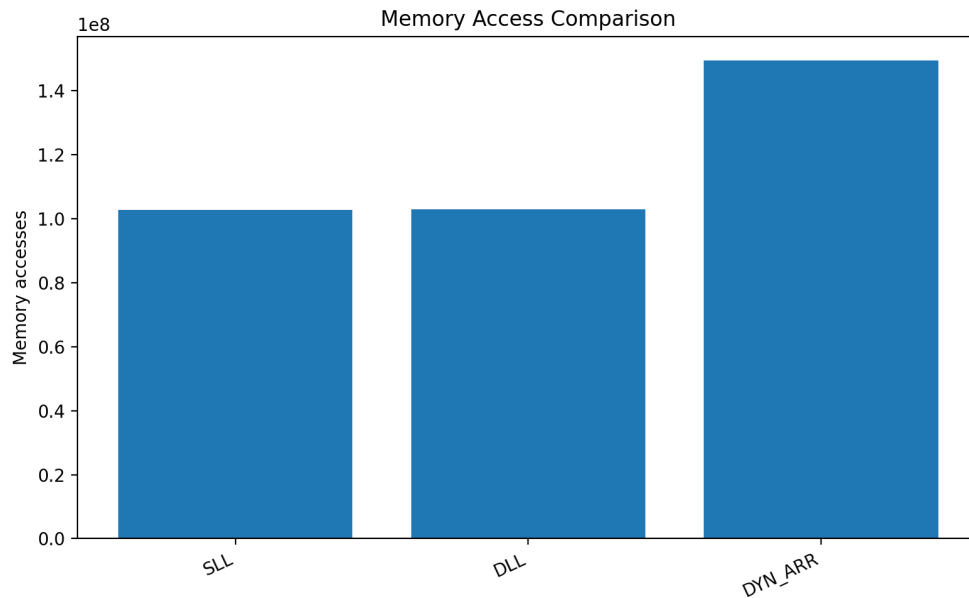
Τα τελικά αποτελέσματα για όλες τις υλοποιήσεις παρουσιάζονται στον παρακάτω πίνακα:

qHead	memory accesses	memory footprint (KB)
SLL	102,802,733	25.23
DLL	102,982,151	30.13
DYN.ARR	149,452,450	21.91

Table 2: Memory accesses και memory footprint για διαφορετικές υλοποιήσεις του qHead.

Δ)

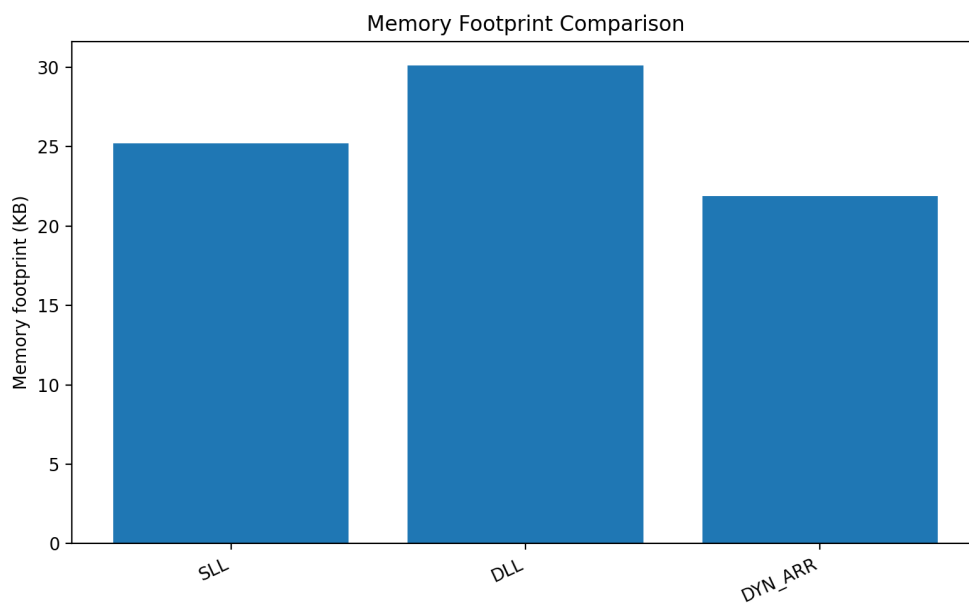
Στο παρακάτω διάγραμμα παρουσιάζουμε τα memory accesses για τις διαφορετικές υλοποιήσεις του qHead:



Παρατηρούμε ότι με μικρή διαφορά τα λιγότερα memory accesses έχει η υλοποίηση με απλά συνδεδεμένη λίστα σε σχέση με την διπλά συνδεδεμένη λίστα και τον μέγιστο αριθμό memory accesses παρατηρούμε για την υλοποίηση με δυναμικό πίνακα.

E)

Στο παρακάτω διάγραμμα παρουσιάζουμε τα memory footprints για τις διαφορετικές υλοποιήσεις:



Όπως φαίνεται λογικό από την μορφή των υλοποιήσεων στις δομές δεδομένων το μικρότερο memory footprint

έχει η υλοποίηση με δυναμικό πίνακα, ακολουθεί η απλά συνδεδεμένη λίστα και το μεγαλύτερο memory footprint έχει η διπλά συνδεδεμένη λίστα (για τους ίδιους λόγους με την 1η άσκηση).